

YOLOv8 object detection on the AMD Versal VCK190

Hardware implementation, measurement and comparison of four models
COCO val2017, 5000 images · September 2026

1. What this covers

Four stock YOLOv8 detectors — n, s, m and l — were quantized to INT8, compiled for the DPU on a Versal VCK190 evaluation board, and run over the whole of COCO val2017. This report records how each one was built, what it scored, how fast it ran, how much power it drew, and how much memory it needed. The last section compares them against each other.

Two things are worth stating up front because they shape everything else. First, the DPU cannot execute the SiLU activation that YOLOv8 ships with, so every model had its activation replaced with HardSwish before quantization. That substitution, not the 8-bit arithmetic, is responsible for most of the accuracy that was lost. Second, the board draws about 13 W whether it is working or not, and inference adds well under one watt on top, so energy per image is governed by how long the board is switched on rather than by which model is loaded.

Model	mAP@0.5	mAP@0.5:0.95	images/s	power (W)	energy (J/img)
yolov8n	0.445271	0.291348	14.38	13.70	0.953
yolov8s	0.542305	0.371572	15.86	14.00	0.883
yolov8m	0.600114	0.425569	14.34	14.01	0.977
yolov8l	0.623348	0.446702	12.45	13.81	1.110

Headline figures. Full tables follow, and the appendix carries every metric collected.

2. Hardware and software

The board is a VCK190 evaluation board, silicon revision v2, with 8 GiB of DRAM. It boots PetaLinux 2022.2 from an SD card and runs the Vitis AI runtime. The DPU is a DPUCVDX8G, configured as ISA3_C32B6: thirty-two channels wide with a batch of six, clocked at 333 MHz. All four models run on this one DPU configuration; the hardware does not change between them.

Item	Value
Board	Xilinx Versal VCK190 evaluation board, rev A, silicon v2, 8 GiB DRAM
DPU	DPUCVDX8G_ISA3_C32B6, batch 6, 333 MHz
DPU fingerprint	433190037845252193 (identical for all four models)
Operating system	PetaLinux 2022.2, kernel 5.15.36-xilinx-v2022.2, aarch64

Runtime	VART 3.0.0, XIR 3.0
Board Python	3.9 with NumPy 1.26.4, OpenCV 4.5.2, pycocotools
Toolchain	Vitis AI 3.0 PyTorch container (vai_q_pytorch, vai_c_xir)
Host link	USB gigabit Ethernet, static 192.168.1.10/24 on the board

3. The four models

These are the stock Ultralytics release weights, dated December 2022, unmodified. Each one detects the standard 80 COCO classes at 640×640 input, with an anchor-free head that predicts box edges as a distribution over 16 bins per side. Class ordering was checked against the standard COCO list on all four before any work started.

Model	params (M)	conv layers	GOP/image	float mAP@0.5	float mAP@0.5:0.95
yolov8n	3.16	57	8.764	0.5187	0.3681
yolov8s	11.17	57	28.641	0.6106	0.4440
yolov8m	25.90	77	79.005	0.6654	0.4979
yolov8l	43.69	97	165.255	0.6916	0.5244

Float scores are the PC reference for the same weights, used throughout as the ceiling the board is measured against.

4. How the models were put on the board

4.1 Splitting the network between DPU and CPU

The DPU runs convolutions. It does not run the arithmetic that turns the head's raw output into boxes — the softmax over the distance bins, the sigmoid on the class scores, and non-maximum suppression are not fixed-function convolution work. So the network was cut at the head's output convolutions. The DPU produces three raw feature maps, at strides 8, 16 and 32, each with 144 channels; the host reads those and does the rest.

144 channels is four box sides $\times$ 16 distribution bins, plus 80 class scores. At 640 input that gives tensors of 80×80 , 40×40 and 20×20 .

Leaving the decode on the DPU is possible and an earlier project did it, but it splits the compiled graph in two. Moving the whole decode to the host leaves one DPU block with nothing in the middle to break it up. That is why this build compiles to a single subgraph where the earlier one produced two.

4.2 The activation problem

YOLOv8 uses SiLU, $x \cdot \text{sigmoid}(x)$. The compiler has no DPU mapping for it. Left alone it shatters the graph into dozens of fragments, and worse, the board carries no runtime library for the operator, so such a build compiles and then refuses to run.

HardSwish is close in shape and the DPU implements it directly. Every SiLU was replaced with HardSwish before quantization — 57 of them in yolov8n and yolov8s, 77 in yolov8m, 97 in yolov8l. This is an approximation, and section 6 measures what it costs. No retraining was done after the swap.

A subtlety that catches people: Ultralytics declares the activation as a class attribute shared by every convolution, so counting activation modules in the model returns one, not fifty-seven. The count that matters is the number of convolution layers, since each applies the activation in its own forward pass.

4.3 Quantization

vai_q_pytorch was run in two passes. The first walks calibration images and records the range each tensor covers; the second exports the INT8 model. Calibration used 200 images taken at even stride through a 4000-image split of val2017 that is disjoint from the 1000 images held back, so the calibration set is not simply the first 200 files.

Weights and activations are 8-bit, with a per-tensor power-of-two scale. The input tensor arrives as int8 with a fixed-point scale of 64; the three outputs come back with a scale of 0.25.

4.4 Compilation

vai_c_xir compiled each INT8 model against the VCK190 architecture file. All four produced the same structure: one DPU subgraph, and three small CPU subgraphs that do nothing but convert fixed-point outputs to float. That conversion operator ships with the board, so there is no missing-library problem at runtime.

Model	xmodel (MiB)	DPU subgraphs	CPU operators	DPU ops	microcode (KiB)
yolov8n	3.39	1	fix2float x3	244	145.5
yolov8s	11.06	1	fix2float x3	244	186.9

yolov8m	25.36	1	fix2float x3	310	393.6
yolov8l	42.69	1	fix2float x3	376	775.4

A single DPU subgraph matters for speed as well as tidiness. With one block the runtime can use the plain VART runner and hand the DPU a full batch of six frames per call. A fragmented graph forces a whole-graph runner that bounces between DPU and CPU at every boundary.

4.5 Decoding on the host

The host turns the three raw tensors into boxes. For each cell it takes the 64 box channels as four groups of 16, applies a softmax within each group and takes the expected value, which gives the distance from the cell centre to each of the four box edges in cell units. Multiplying by the stride and offsetting from the cell centre gives pixel coordinates. Class scores are a straight sigmoid; YOLOv8 has no separate objectness channel.

Before any of this was run on hardware it was checked against Ultralytics' own decode on identical features. Boxes agreed to 6×10^{-5} pixels and scores to 6×10^{-8} on all four models, which is floating-point noise.

Cost on the host is small. At the confidence threshold used for scoring, decode and NMS take about 4.8 ms per image, against a DPU that needs roughly 45 ms. The saving comes from rejecting cells on their strongest class score before doing any softmax work, so the expensive step only touches the thousand or so candidates that survive.

4.6 Preprocessing

Images are letterboxed: scaled to fit 640×640 with the aspect ratio preserved and the remainder filled with grey. The alternative, squashing to a square, distorts every object and costs real accuracy — about 0.021 mAP in float and 0.047 after quantization on yolov8n. Boxes are mapped back afterwards by subtracting the padding and dividing by the scale.

The padding subtracted on the way back must be the integer number of pixel rows and columns actually added, not the unrounded half-pixel value. Using the latter leaves every box shifted by up to half a pixel on roughly half of COCO's aspect ratios. It is invisible at a loose IoU threshold and quietly erodes the strict ones.

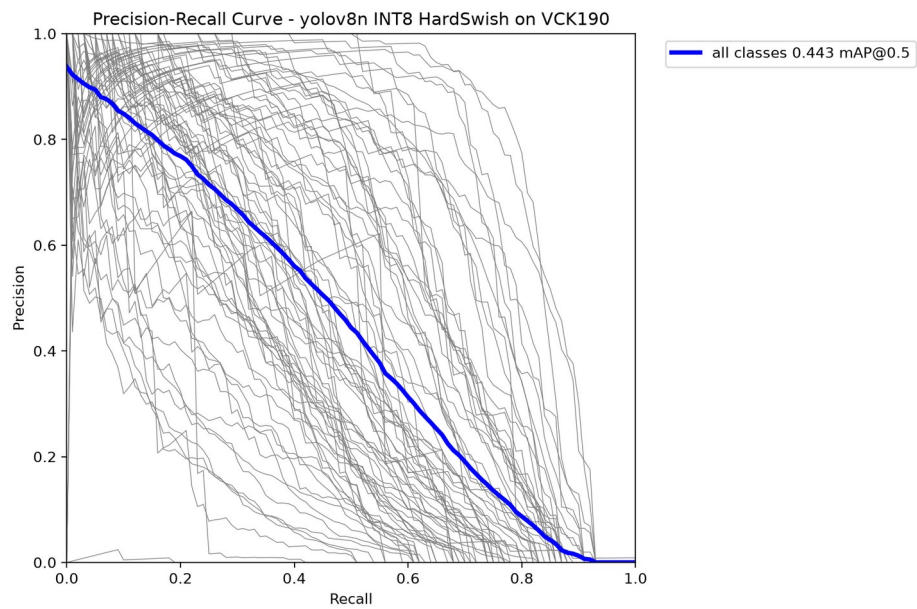
5. How each model performed

Every model was run over all 5000 validation images with a confidence threshold of 0.001, IoU 0.45 for NMS and at most 300 detections per image. Scoring is pycocotools. Precision, recall and F1 are quoted at the confidence that maximises F1, which is the threshold you would actually deploy at; mAP integrates across all thresholds and says nothing about any single operating point.

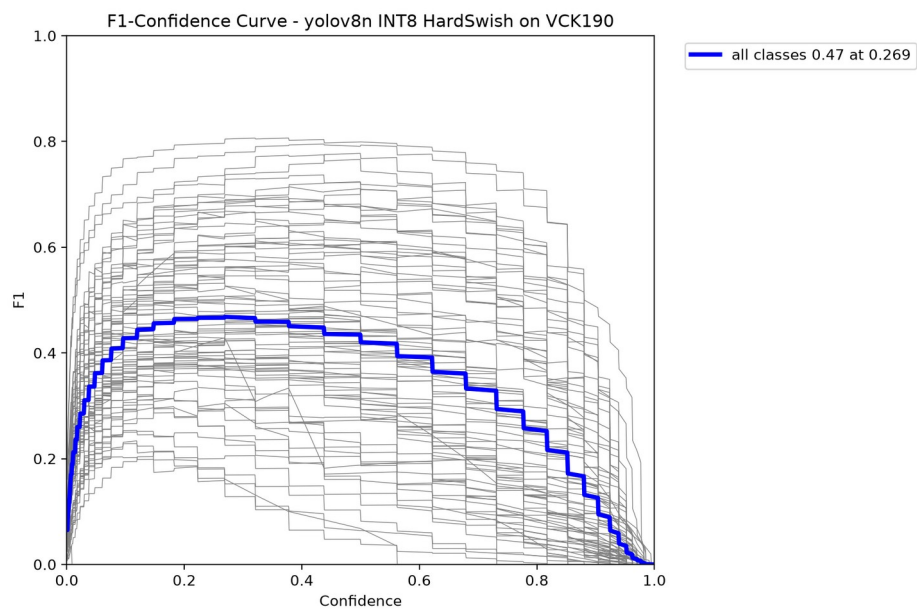
5.1 yolov8n

The smallest model, and the one that fits the hardware worst. It occupies only 87.4% of the DPU array — the rest is padding — so it runs no faster than yolov8m despite doing a ninth of the arithmetic. It is also the only model whose best operating point sits below 0.27 confidence, a sign that its class scores are poorly separated after quantization.

Accuracy	value	Hardware	value
mAP@0.5	0.445271	Throughput, end to end	14.38 img/s
mAP@0.5:0.95	0.291348	Throughput, DPU only	22.16 img/s
mAP@0.75	0.307229	Latency per image	69.54 ms
AR@100	0.494916	Runtime, 5000 images	347 s
Precision	0.5889	Power, idle	13.02 W
Recall	0.4123	Power, under load	13.70 W
F1	0.4682	Power, inference only	0.69 W
Operating confidence	0.269	Energy per image	0.953 J
Detections (5000 images)	794,255	Efficiency	1.05 img/s/W
Second scorer, mAP@0.5	0.4430	Compute	8.764 GOP/image
Retained vs float @0.5	85.8 %	Array utilisation	87.4 %
Retained vs float @0.5:0.95	79.1 %	DDR footprint	16.43 MiB



yolov8n: precision against recall. Grey lines are the 80 individual classes, the bold line their mean.

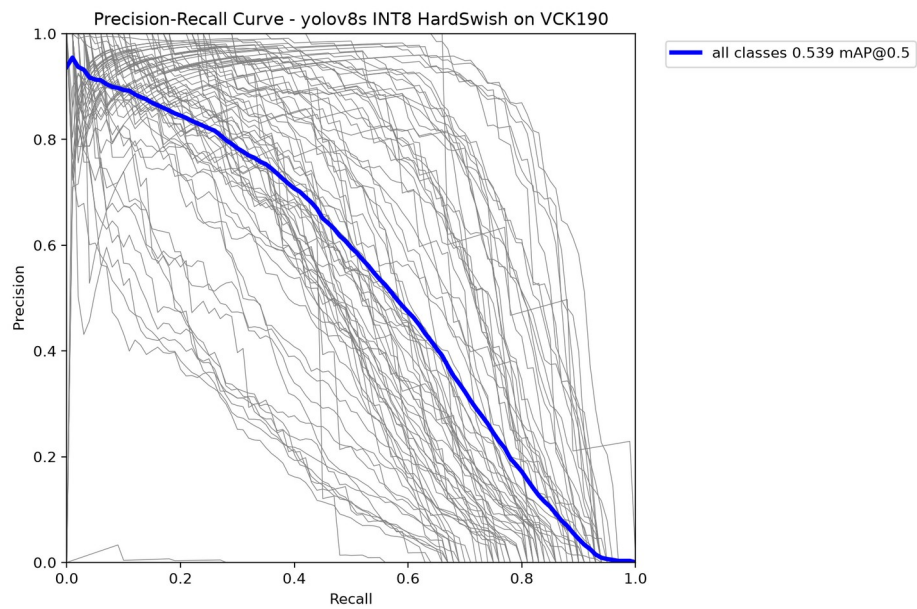


yolov8n: F1 against confidence. The peak marks the operating point quoted in the table above.

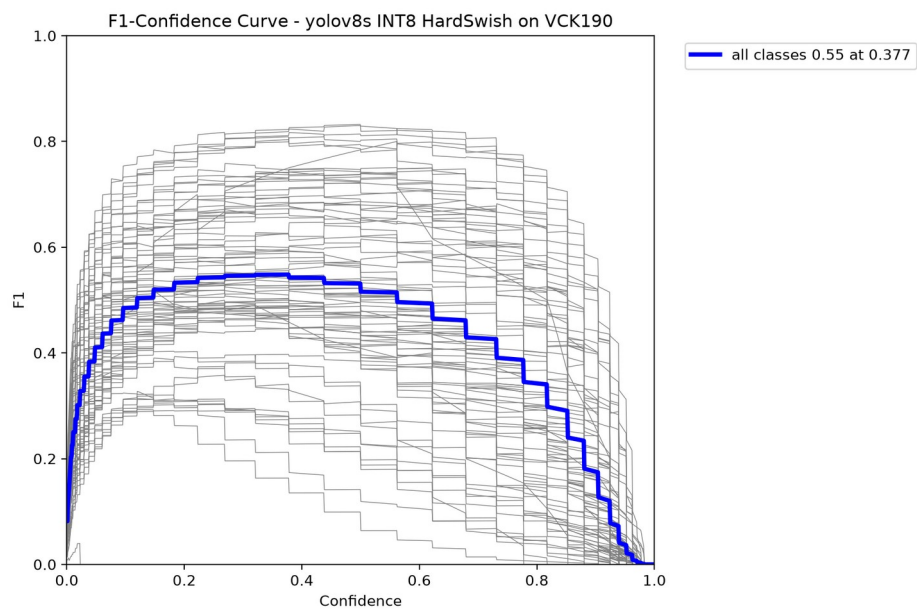
5.2 yolov8s

The best-balanced of the four on this board. It is the fastest, the most energy-efficient, and 22% more accurate than yolov8n. It fills the array well at 95.5% and shares the same 57 convolution layers as yolov8n, so the extra work is in wider channels rather than more layers — which is exactly the shape the DPU handles well.

Accuracy	value	Hardware	value
mAP@0.5	0.542305	Throughput, end to end	15.86 img/s
mAP@0.5:0.95	0.371572	Throughput, DPU only	22.24 img/s
mAP@0.75	0.393132	Latency per image	63.05 ms
AR@100	0.560285	Runtime, 5000 images	315 s
Precision	0.6523	Power, idle	13.27 W
Recall	0.4906	Power, under load	14.00 W
F1	0.5482	Power, inference only	0.73 W
Operating confidence	0.377	Energy per image	0.883 J
Detections (5000 images)	637,641	Efficiency	1.13 img/s/W
Second scorer, mAP@0.5	0.5391	Compute	28.641 GOP/image
Retained vs float @0.5	88.8 %	Array utilisation	95.5 %
Retained vs float @0.5:0.95	83.7 %	DDR footprint	34.40 MiB



yolov8s: precision against recall. Grey lines are the 80 individual classes, the bold line their mean.

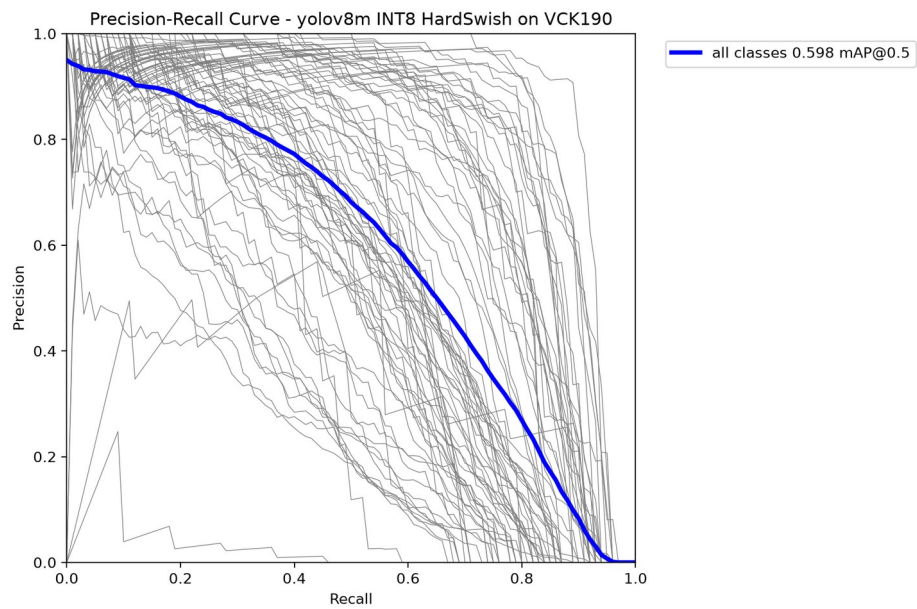


yolov8s: F1 against confidence. The peak marks the operating point quoted in the table above.

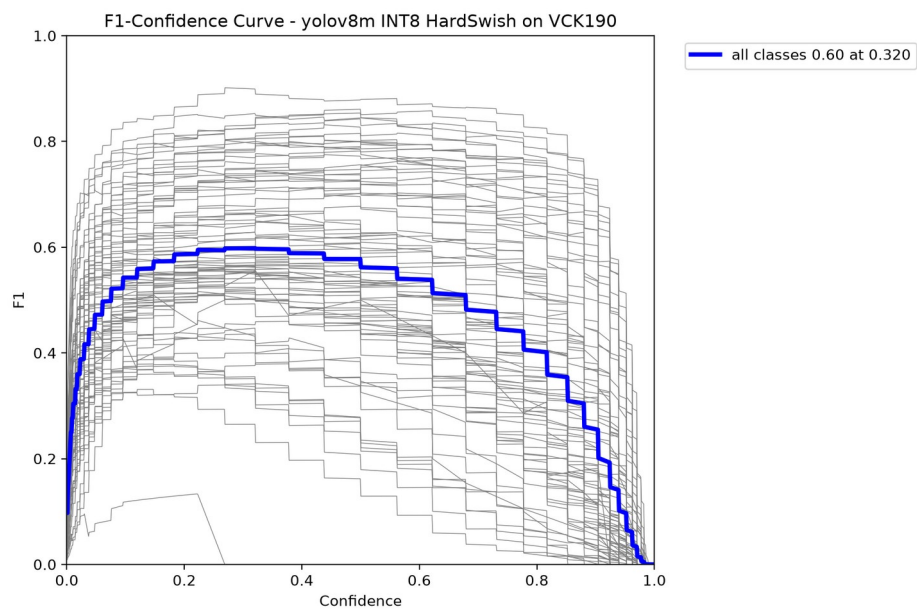
5.3 yolov8m

Deeper than s, with 77 convolution layers, and the accuracy gain is solid. Throughput falls back to roughly yolov8n's level, but for three times the accuracy per image. Array utilisation is 94.9%.

Accuracy	value	Hardware	value
mAP@0.5	0.600114	Throughput, end to end	14.34 img/s
mAP@0.5:0.95	0.425569	Throughput, DPU only	20.69 img/s
mAP@0.75	0.456461	Latency per image	69.74 ms
AR@100	0.605190	Runtime, 5000 images	348 s
Precision	0.6621	Power, idle	13.30 W
Recall	0.5616	Power, under load	14.01 W
F1	0.5981	Power, inference only	0.71 W
Operating confidence	0.320	Energy per image	0.977 J
Detections (5000 images)	773,508	Efficiency	1.02 img/s/W
Second scorer, mAP@0.5	0.5976	Compute	79.005 GOP/image
Retained vs float @0.5	90.2 %	Array utilisation	94.9 %
Retained vs float @0.5:0.95	85.5 %	DDR footprint	59.98 MiB



yolov8m: precision against recall. Grey lines are the 80 individual classes, the bold line their mean.

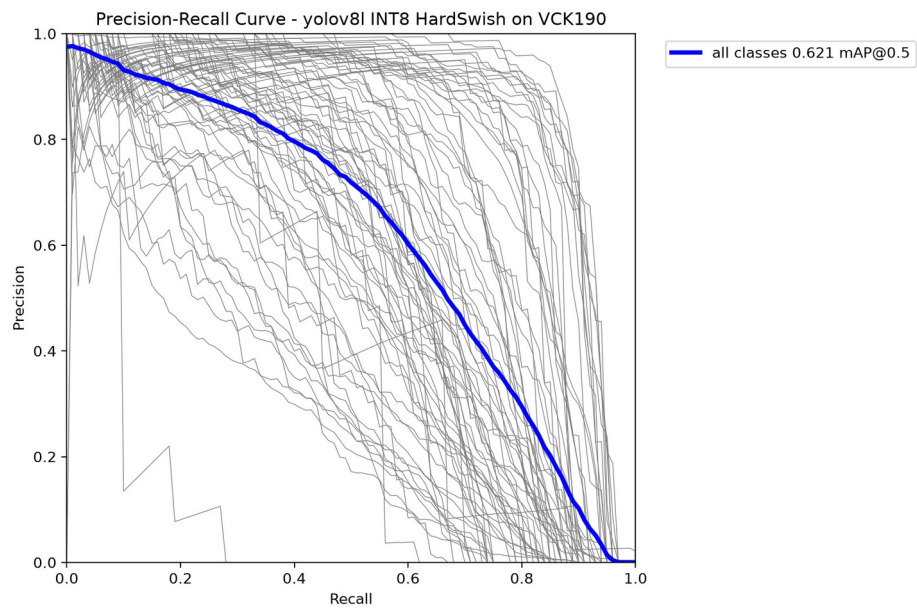


yolov8m: F1 against confidence. The peak marks the operating point quoted in the table above.

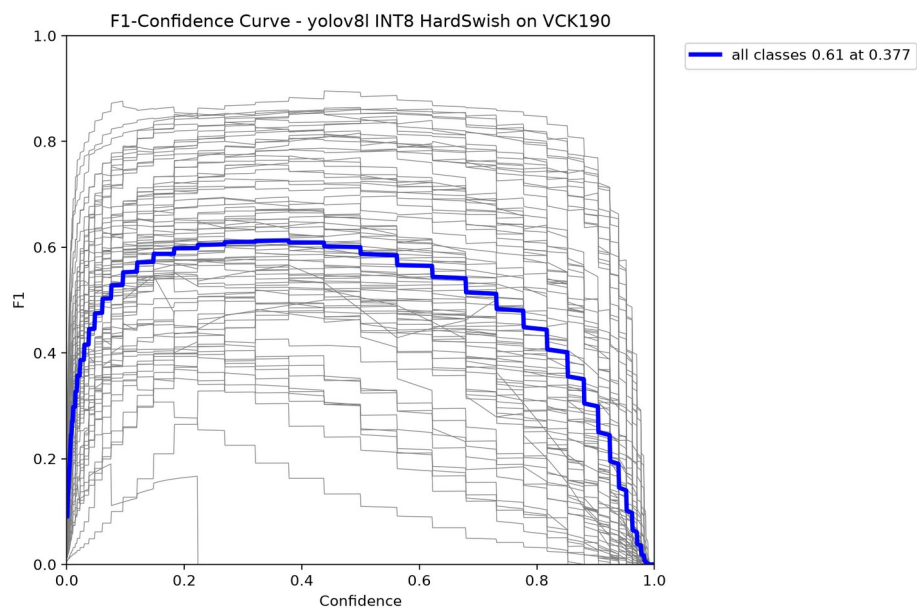
5.4 yolov8l

The most accurate and the slowest. It uses the DPU array best of all at 98.2%, and draws the most power under load at 1.01 W above idle, which is consistent with it keeping the array busiest. Its DDR footprint of 93.8 MiB is the largest but still modest against 8 GiB.

Accuracy	value	Hardware	value
mAP@0.5	0.623348	Throughput, end to end	12.45 img/s
mAP@0.5:0.95	0.446702	Throughput, DPU only	15.04 img/s
mAP@0.75	0.480741	Latency per image	80.32 ms
AR@100	0.618074	Runtime, 5000 images	401 s
Precision	0.7295	Power, idle	12.81 W
Recall	0.5559	Power, under load	13.81 W
F1	0.6128	Power, inference only	1.01 W
Operating confidence	0.377	Energy per image	1.110 J
Detections (5000 images)	957,300	Efficiency	0.90 img/s/W
Second scorer, mAP@0.5	0.6211	Compute	165.255 GOP/image
Retained vs float @0.5	90.1 %	Array utilisation	98.2 %
Retained vs float @0.5:0.95	85.2 %	DDR footprint	93.83 MiB



yolov8l: precision against recall. Grey lines are the 80 individual classes, the bold line their mean.



yolov8l: F1 against confidence. The peak marks the operating point quoted in the table above.

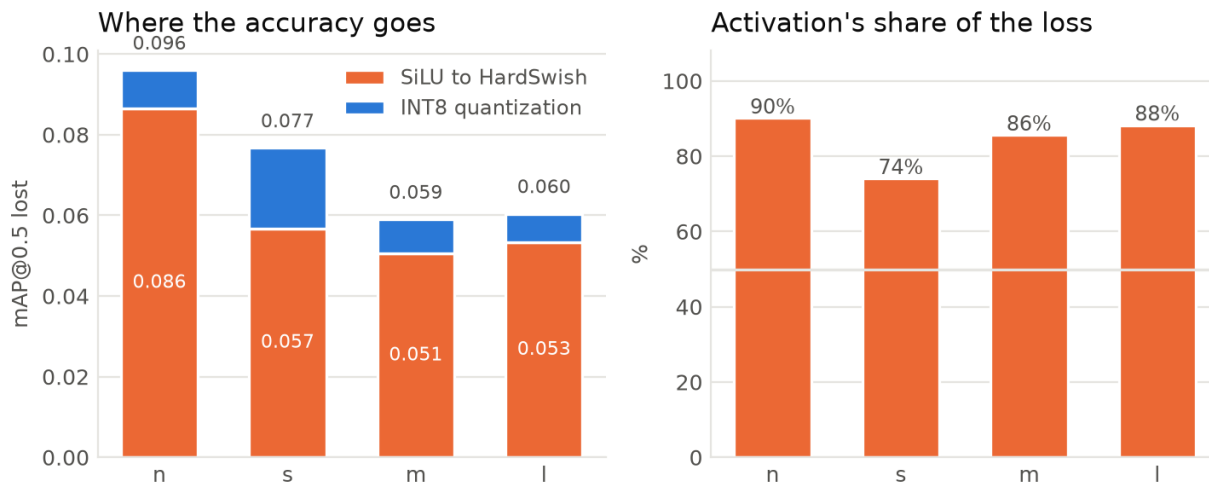
6. What quantization costs, and what the activation costs

Going from a float model on a PC to an INT8 model on the DPU involves two separate changes, and they are easy to confuse. The activation is replaced with HardSwish, and the arithmetic is reduced to 8 bits. Comparing the float PC score against the board score measures both at once and cannot tell you which one hurt.

To separate them, each model was run three ways on an identical 500-image subset, with letterbox preprocessing throughout, changing one thing at a time: float with the original SiLU, float with HardSwish substituted, and INT8 with HardSwish. The step between the first two is the activation; the step between the last two is quantization.

Model	float SiLU	float HardSwish	INT8 HardSwish	activation	quantization	activation share
yolov8n	0.5674	0.4809	0.4714	-0.0865	-0.0095	90%
yolov8s	0.6448	0.5880	0.5681	-0.0568	-0.0199	74%
yolov8m	0.6940	0.6434	0.6349	-0.0505	-0.0085	86%
yolov8l	0.7222	0.6689	0.6618	-0.0533	-0.0071	88%

mAP@0.5 on the same 500 images. The two delta columns sum to the total drop. The last column is the activation's share of it.



Each model loses far more to the activation substitution than to 8-bit arithmetic.

The pattern is the same for every model: the activation substitution costs several times what quantization costs. This matters because it points the remedy in a particular direction. Tightening the quantizer — more calibration images, per-channel scales, percentile clipping — works on the smaller of the two problems. The larger one is that the weights were trained for SiLU and then asked to run with HardSwish, with no opportunity to adapt.

6.1 A closer look at yolov8n

For yolov8n the experiment was taken one step further. SiLU was rewritten as its exact identity, x multiplied by $\text{sigmoid}(x)$, which is numerically the same function but built from operators the toolchain can handle. Quantizing that gives an INT8

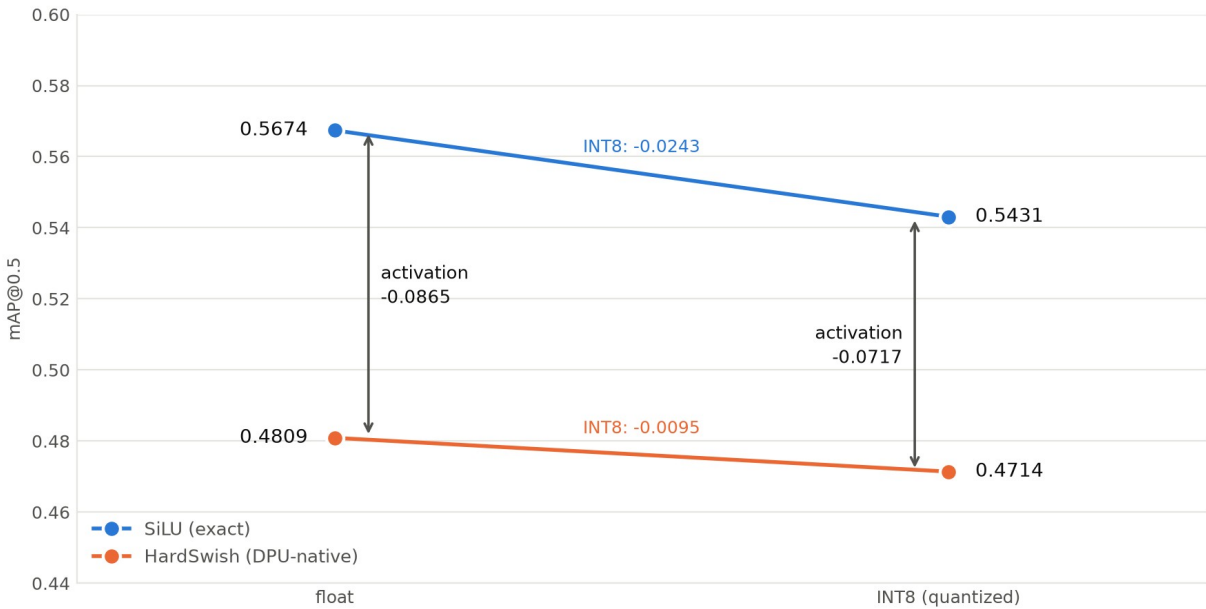
model with the activation held exactly right, so the two factors can be crossed:

yolov8n, mAP@0.5	float	INT8	cost of quantization
SiLU (exact)	0.5674	0.5431	-0.0243
HardSwish (approximate)	0.4809	0.4714	-0.0095
cost of activation	-0.0865	-0.0717	total -0.0960

Both routes from 0.5674 down to 0.4714 add up to the same total, which is the arithmetic check that the four numbers are consistent. The two effects interact, so the exact split depends on which you attribute first: the activation costs between 0.0717 and 0.0865, quantization between 0.0095 and 0.0243. Averaging, the activation accounts for roughly 82% of the loss and quantization 18%. Whichever way it is cut, the activation dominates.

What the DPU actually costs - yolov8n

The gap between the lines is the activation swap; each line's slope is INT8.
500-image subset, letterbox, all four points from one pipeline.



The gap between the two lines is the activation substitution; each line's slope is quantization. yolov8n, 500-image subset.

The exact-SiLU model is a measurement tool, not a deployable one. It compiles for YOLOv3 but `vai_c_xir` rejects it for YOLOv8, most likely because of the elementwise multiply landing inside the C2f split-and-concatenate structure. The numbers above come from CPU simulation of the INT8 model, which reproduces the board to within 0.004 mAP where the two can be compared.

The practical conclusion is that a short finetune with HardSwish already in place should recover most of the gap, because the weights only need to re-adapt to a slightly different nonlinearity rather than relearn the task. That needs a GPU, which this machine does not have, so it has not been attempted.

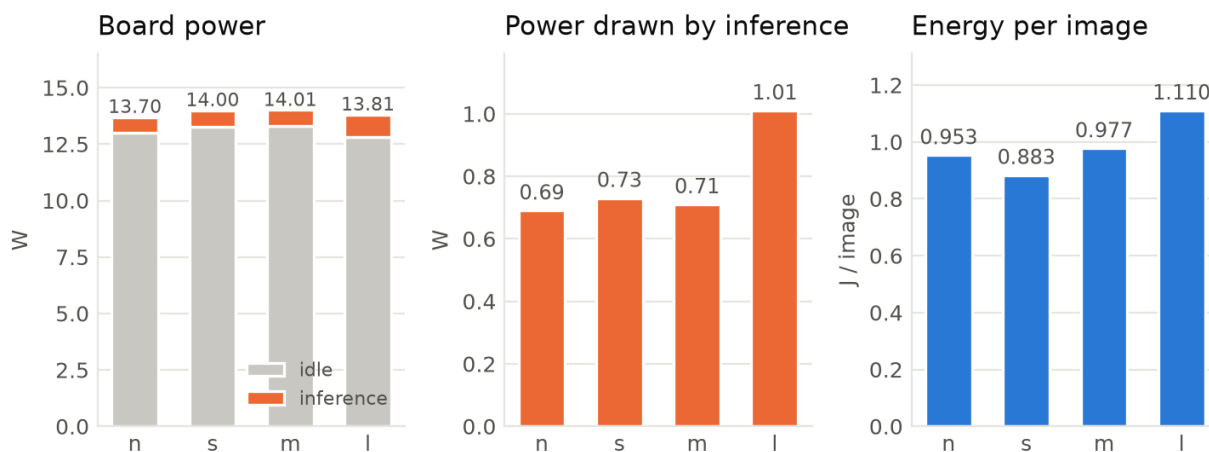
7. Power, current and energy

The VCK190 carries seventeen INA226 monitors on its supply rails, readable through the kernel's hwmon interface. Each reports rail power directly in microwatts. Power was sampled every half second, first with the board idle for twelve seconds, then while the DPU was driven with 400 back-to-back inferences.

Driving the DPU continuously matters. An earlier attempt sampled during an ordinary walk through the dataset, where the DPU sits idle between frames waiting on image loading and decode; average power barely rose above idle and the figure understated the load by more than half. The numbers below come from sustained load.

Polling seventeen I²C sensors is not free: it dragged throughput from 14.4 down to 6.7 images per second on the first attempt. Energy per image is therefore calculated from the throughput measured in a clean run without sampling, not from the rate observed while the sensors were being read.

Model	idle (W)	load (W)	inference (W)	J/img total	J/img inference	img/s/W
yolov8n	13.02	13.70	0.69	0.953	0.048	1.05
yolov8s	13.27	14.00	0.73	0.883	0.046	1.13
yolov8m	13.30	14.01	0.71	0.977	0.050	1.02
yolov8l	12.81	13.81	1.01	1.110	0.081	0.90



Idle dominates completely. The board draws roughly 13 W doing nothing, and running the largest of the four models on top of that adds 1.01 W. For the smallest it is 0.69 W. In other words, between 93% and 95% of the energy spent per image is the board being switched on, not the detector running.

This inverts the usual intuition. Choosing a smaller model to save energy does almost nothing here, because the model was never the thing consuming the energy. What does help is finishing sooner: yolov8s uses the least energy per image of the four, not because it draws less power — it draws slightly more than yolov8n — but because it gets through more images per second and so spends less time per image paying the 13 W standing charge.

7.1 Per-rail breakdown

Rail-level readings, taken while yolov8n ran, show where the extra power goes. Most rails do not move at all. Four do:

Rail	idle (W)	load (W)	change (W)
hwmon1	5.188	5.403	+0.215
hwmon5	0.163	0.393	+0.230
hwmon12	1.102	1.287	+0.185
hwmon0	3.620	3.674	+0.054
hwmon6	1.500	1.500	0.000
hwmon15	0.406	0.406	0.000
other 11 rails	0.938	0.939	+0.001
total	13.02	13.70	+0.68

hwmon5 more than doubles, from 0.163 W to 0.393 W, which is the largest proportional change of any rail and is almost certainly the programmable-logic supply feeding the DPU. hwmon1 is the biggest rail in absolute terms and contributes a similar amount of the increase. The remaining eleven rails are flat to within a milliwatt.

Rail voltages and currents were not recorded, only rail power. The same INA226 devices expose voltage and current separately and the sampling script could read them, but the board was taken offline before that was done. The rails are also identified only by their hwmon index here; mapping those to named supplies needs the board's device tree, which was not consulted.

8. Area and memory

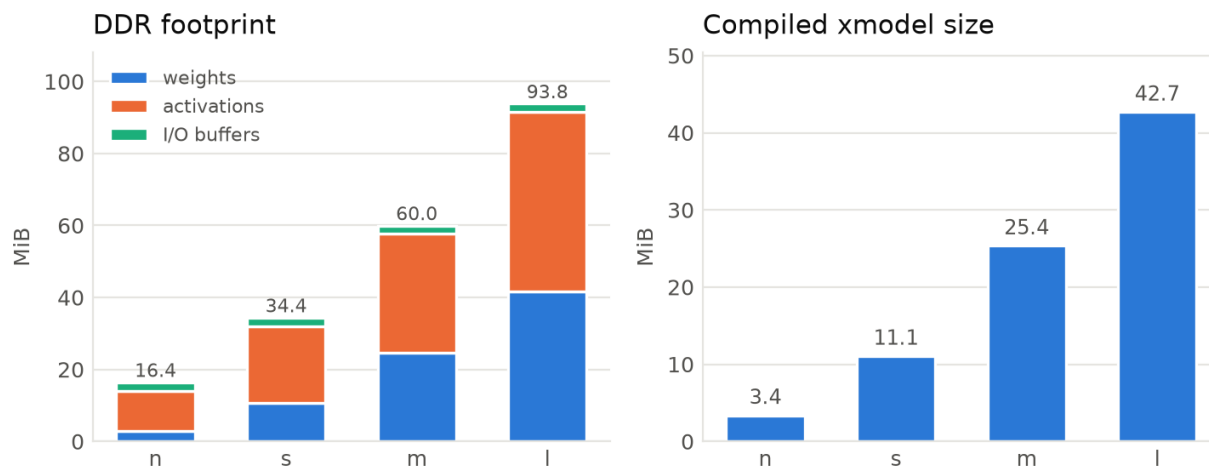
Asking how much area each model occupies needs a distinction that is easy to skip over. On an FPGA, area normally means logic resources — lookup tables, flip-flops, block RAM, DSP slices. Those belong to the DPU itself, which is a fixed piece of hardware compiled into the device once. All four models run on that same DPU. Loading yolov8l instead of yolov8n does not change a single lookup table.

The evidence is in the compiled files: every one of the four carries the same DPU fingerprint, 433190037845252193. They are addressed to the same hardware. A per-model table of LUT and DSP counts would be one number repeated four times, and it would be misleading to present it as though it varied.

The actual utilisation figures for this DPU configuration come from the Vivado implementation report for the platform. This project runs a prebuilt Vitis AI board image and never ran implementation, so those numbers are not available here and have not been guessed at. Obtaining them means rebuilding the platform from the DPU reference design.

What does vary per model is memory: how much DDR the weights occupy, how much the intermediate feature maps need, and how large the compiled program is. That is the footprint that decides whether a model fits and how much bandwidth it costs.

Model	weights (MiB)	activations (MiB)	I/O buffers (MiB)	DDR total (MiB)	xmodel (MiB)	microcode (KiB)
yolov8n	3.01	11.10	2.32	16.43	3.39	145.5
yolov8s	10.64	21.44	2.32	34.40	11.06	186.9
yolov8m	24.69	32.96	2.32	59.98	25.36	393.6
yolov8l	41.65	49.85	2.32	93.83	42.69	775.4



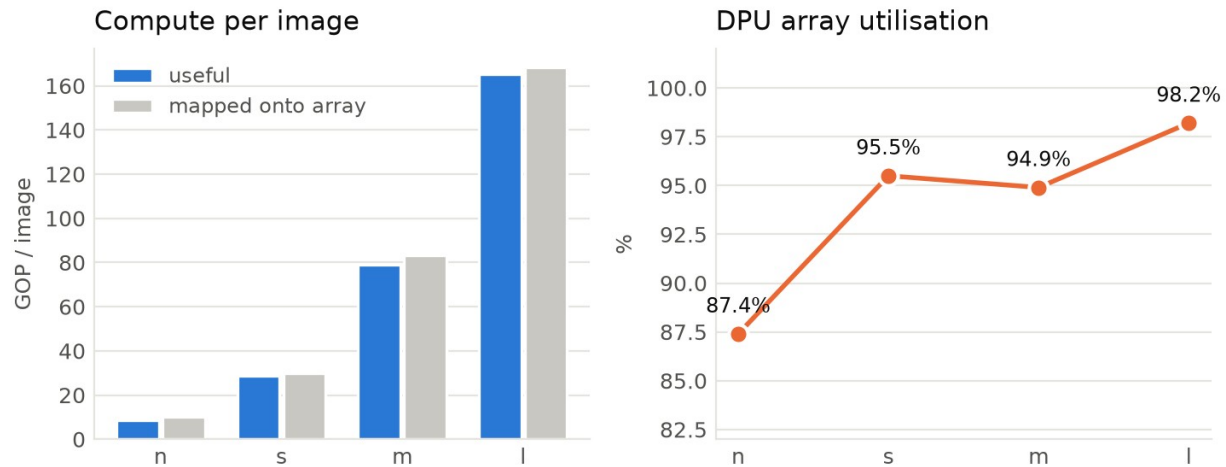
The input and output buffers are identical across all four, as they must be: the input is always 640×640×3 and the output always 8400 positions of 144 channels. Weight storage tracks parameter count. Activation storage grows more slowly than weights do, which is why the ratio shifts from roughly one part weights to four parts activations in yolov8n, to something near parity in yolov8l.

Against 8 GiB of board DRAM none of these are large. The largest, yolov8l at 93.8 MiB, uses just over one percent of it. Memory is not the constraint on this platform.

8.1 How well each model fills the DPU

There is a second sense of area worth reporting, which is how much of the DPU's compute array a model actually keeps busy. The compiler records both the useful work in a model and the work as mapped onto the array; the difference is padding, where channel counts do not divide evenly into the array geometry.

Model	useful GOP/img	mapped GOP/img	utilisation (%)	achieved GOP/s	DPU operators
yolov8n	8.764	10.031	87.4	126.0	244
yolov8s	28.641	29.985	95.5	454.2	244
yolov8m	79.005	83.248	94.9	1132.9	310
yolov8l	165.255	168.270	98.2	2057.4	376



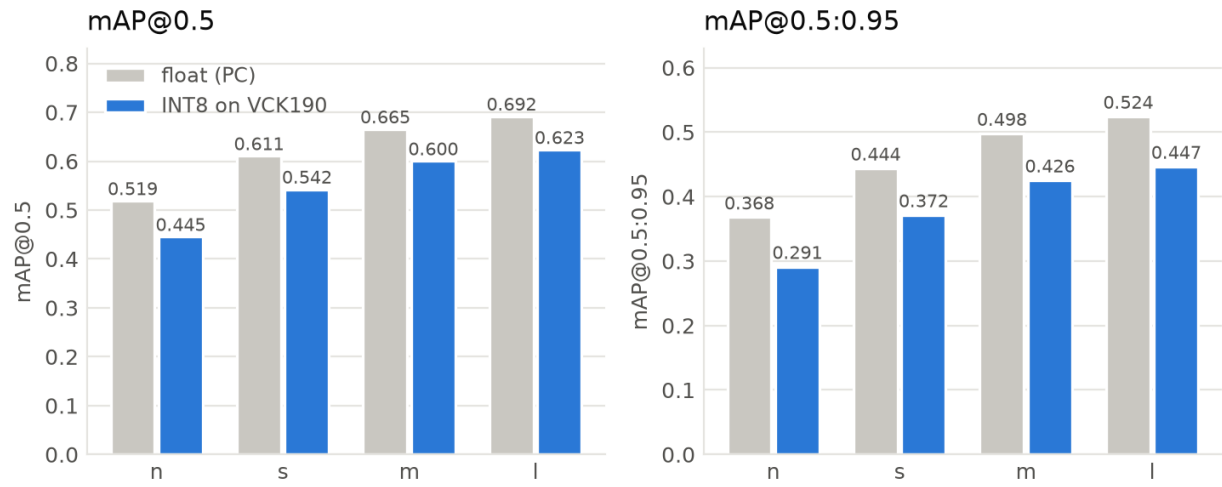
yolov8n is the worst fit at 87.4%: one operation in eight that the array performs for it is wasted on padding. yolov8l is the best at 98.2%. This is the main reason yolov8n runs no faster than yolov8m despite needing a ninth of the arithmetic — the small model is too narrow to fill a 32-channel array, so it cannot convert its lighter workload into proportionally more speed.

9. The four compared

Everything measured, side by side.

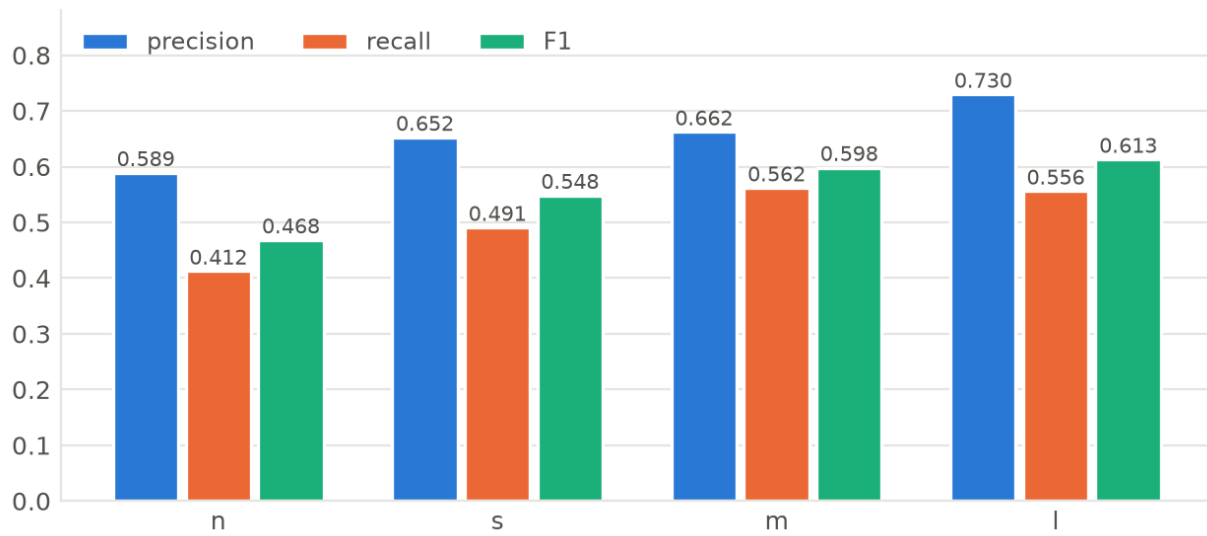
Metric	yolov8n	yolov8s	yolov8m	yolov8l
mAP@0.5	0.445271	0.542305	0.600114	0.623348
mAP@0.5:0.95	0.291348	0.371572	0.425569	0.446702
mAP@0.75	0.307229	0.393132	0.456461	0.480741
AR@100	0.494916	0.560285	0.605190	0.618074
Precision	0.5889	0.6523	0.6621	0.7295
Recall	0.4123	0.4906	0.5616	0.5559
F1	0.4682	0.5482	0.5981	0.6128
Retained vs float @0.5 (%)	85.8	88.8	90.2	90.1
Throughput (img/s)	14.38	15.86	14.34	12.45
DPU-only throughput (img/s)	22.16	22.24	20.69	15.04
Latency (ms)	69.54	63.05	69.74	80.32
Power, load (W)	13.70	14.00	14.01	13.81
Power, inference (W)	0.69	0.73	0.71	1.01
Energy (J/img)	0.953	0.883	0.977	1.110
Efficiency (img/s/W)	1.05	1.13	1.02	0.90
Parameters (M)	3.16	11.17	25.90	43.69
xmodel (MiB)	3.39	11.06	25.36	42.69
DDR total (MiB)	16.43	34.40	59.98	93.83
Compute (GOP/img)	8.764	28.641	79.005	165.255
Array utilisation (%)	87.4	95.5	94.9	98.2

Achieved (GOP/s)	126.0	454.2	1132.9	2057.4

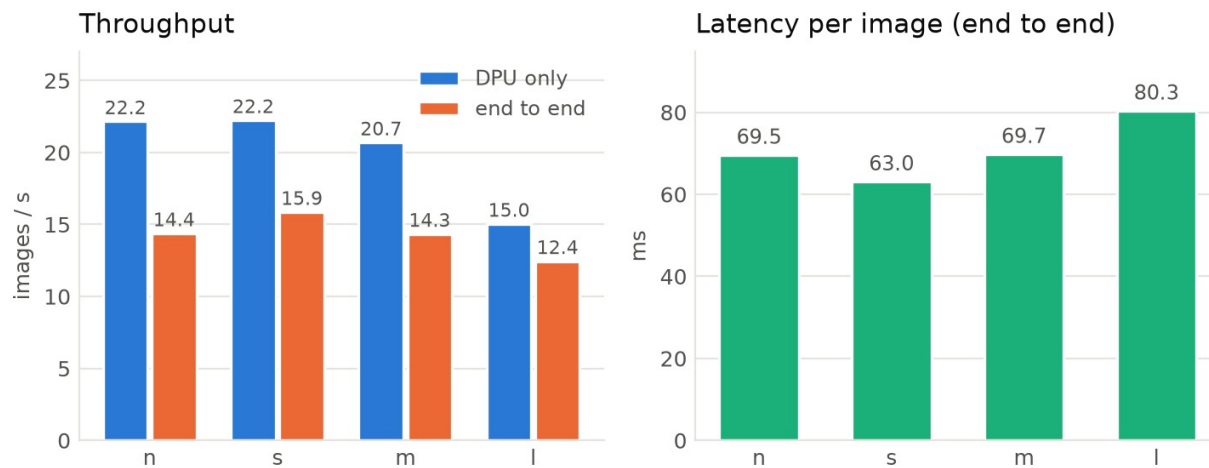


Board INT8 against the float ceiling. The gap narrows as models get larger.

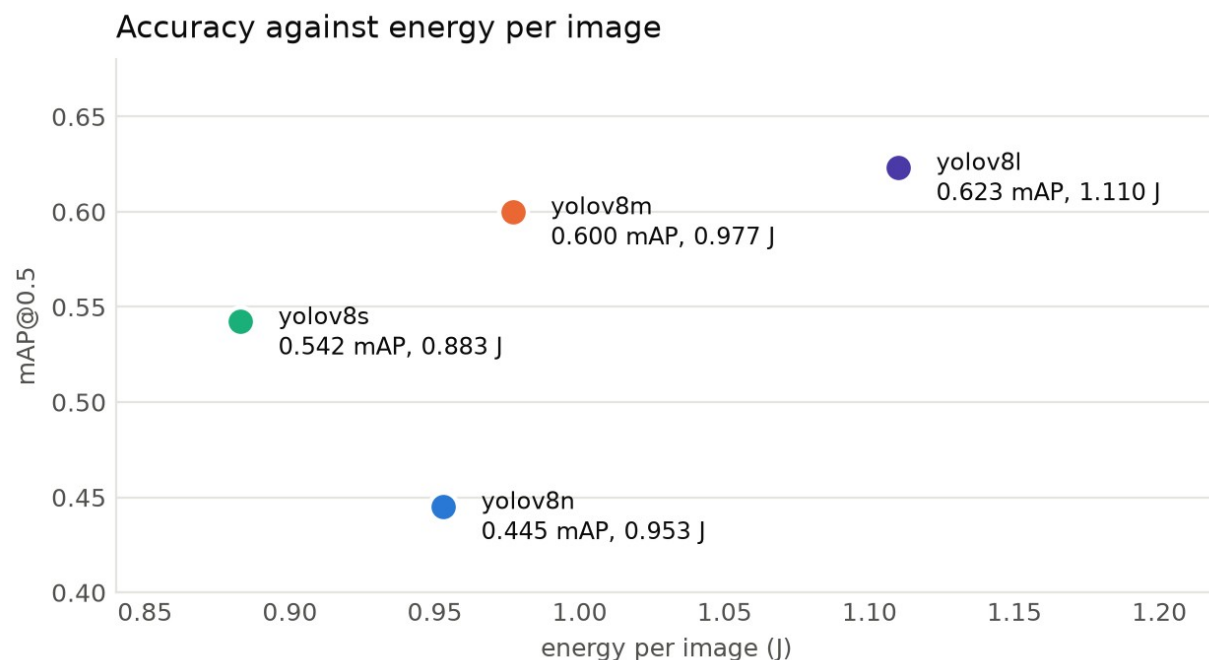
Precision, recall and F1 at the best-F1 operating point



Precision, recall and F1 at each model's best-F1 confidence.



DPU-only throughput against end-to-end, and the resulting latency.



The deployment decision in one picture.

9.1 What the numbers say

yolov8n is dominated. yolov8s beats it on every axis at once: 22% more accurate, faster in both end-to-end and DPU-only terms, and lower energy per image. There is no operating point at which yolov8n is the right answer on this board. The reason is the array utilisation figure — yolov8n is too small to fill the hardware, so its lighter workload buys nothing, while its accuracy penalty is real.

Accuracy is cheap in energy terms. Going from yolov8n to yolov8l gains 0.178 mAP@0.5, a 40% relative improvement, for 0.157 J more per image — about 16% more energy. That trade is only available because idle power dwarfs inference power; on a platform where the accelerator dominated the power budget it would look very different.

Retention improves with size. The board keeps 85.8% of yolov8n's float mAP@0.5 but 90.1% of yolov8l's. Larger models carry more redundancy and survive both the activation substitution and 8-bit arithmetic better. The same ordering holds on

the stricter mAP@0.5:0.95, from 79.1% to 85.2%.

Recall is the weak side. Precision at the operating point ranges from 0.589 to 0.730, recall from 0.412 to 0.562. Every model finds fewer objects than it correctly labels the ones it does find. If the application cares more about missing objects than about false alarms, the operating confidence should be set lower than the best-F1 point quoted here.

9.2 Which to use

For most purposes on this board, yolov8s. It is the efficiency optimum and beats the smaller model outright. If accuracy is what matters and 12.5 images per second is enough, yolov8l costs little more energy and gives 0.623 mAP@0.5. yolov8m sits between them without a strong argument for itself, being no faster than yolov8n and less accurate than yolov8l. yolov8n has no case on this hardware, though it would on a smaller accelerator that it could actually fill.

10. Checks, and what is not measured

10.1 Checks that were run

The decode was verified against Ultralytics' own implementation on identical features before any board time was spent, agreeing to 6×10^{-5} pixels on all four models. The letterbox inverse was separately checked against the reference across ten aspect ratios including the awkward half-pixel cases.

Every result was scored twice, by pycocotools and by an independent implementation that shares no matching code with it:

Model	pycocotools mAP@0.5	independent mAP@0.5	difference
yolov8n	0.4452	0.4430	0.0023
yolov8s	0.5423	0.5391	0.0032
yolov8m	0.6001	0.5976	0.0025
yolov8l	0.6233	0.6211	0.0022

They agree within 0.003 on all four and in the same direction each time, which is what you would expect from one scorer matching at a single IoU threshold and the other sweeping ten. The compiled graph was also inspected before deployment to confirm that every CPU operator it needs actually exists on the board.

10.2 Not measured

Rail voltages and currents. Only rail power was captured. The hardware exposes voltage and current separately and reading them is a small change to the sampling script, but the board was taken offline before it was done.

FPGA resource utilisation. Not available for the reasons in section 8: the DPU is a prebuilt platform and implementation was never run here. It is also not a per-model quantity.

Thermal behaviour. No junction temperatures were recorded and no sustained thermal soak was performed. The runs are short enough that throttling is unlikely but it was not checked.

Accuracy after finetuning. Section 6 argues that a HardSwish finetune should recover most of the lost accuracy. That is a prediction from the measured decomposition, not a result. It needs a GPU.

10.3 One unresolved discrepancy

An earlier deployment of the same four models on this board, by a separate project, produced slightly higher mAP@0.5:0.95 than this one, while this one scores higher on mAP@0.5. The gap on the stricter metric is small and grows with model size: 0.0004 on yolov8n, 0.0061 on yolov8l.

A half-pixel error in the letterbox inverse was found and fixed during this work, and was initially assumed to explain it. Re-running all four after the fix recovered only about a quarter of the gap. The remainder is not explained. The most likely

candidates are that the earlier build kept part of the box decode inside the compiled graph, where it runs at different precision, or a difference in how the two letterbox implementations round. Both prediction sets are kept, so this can be settled by comparing box coordinates for matched detections without using the board.

Appendix A. Every metric collected

All values as recorded, per model. Rows marked otherwise are explained in the sections above.

Parameter	Unit	yolov8n	yolov8s	yolov8m	yolov8l
fps	images/s	14.38	15.86	14.34	12.45
fps dpu only	images/s	22.16	22.24	20.69	15.04
latency per image	ms	69.54	63.05	69.74	80.32
runtime 5000 images	s	347	315	348	401
power idle	W	13.02	13.27	13.30	12.81
power under load	W	13.70	14.00	14.01	13.81
power attributable	W	0.69	0.73	0.71	1.01
energy per image	J/image	0.953	0.883	0.977	1.110
energy per image attributable	J/image	0.048	0.046	0.050	0.081
efficiency	images/s/W	1.05	1.13	1.02	0.90
mAP@0.5		0.445271	0.542305	0.600114	0.623348
mAP@0.5:0.95		0.291348	0.371572	0.425569	0.446702
mAP@0.75		0.307229	0.393132	0.456461	0.480741
AR@100		0.494916	0.560285	0.605190	0.618074
mAP@0.5 independent scorer		0.4430	0.5391	0.5976	0.6211
float mAP@0.5		0.5187	0.6106	0.6654	0.6916

float mAP@0.5:0.95		0.3681	0.4440	0.4979	0.5244
accuracy retained@0.5	%	85.8	88.8	90.2	90.1
accuracy retained@0.5:0.95	%	79.1	83.7	85.5	85.2
precision		0.5889	0.6523	0.6621	0.7295
recall		0.4123	0.4906	0.5616	0.5559
f1 score		0.4682	0.5482	0.5981	0.6128
operating confidence		0.269	0.377	0.320	0.377
detections total	count	794255	637641	773508	957300
parameters	M	3.16	11.17	25.90	43.69
xmodel size	MiB	3.39	11.06	25.36	42.69
ddr weights	MiB	3.01	10.64	24.69	41.65
ddr activations	MiB	11.10	21.44	32.96	49.85
ddr input buffer	MiB	1.17	1.17	1.17	1.17
ddr output buffer	MiB	1.15	1.15	1.15	1.15
ddr total	MiB	16.43	34.40	59.98	93.83
microcode size	KiB	145.5	186.9	393.6	775.4
workload	GOP/ image	8.764	28.641	79.005	165.255
workload on arch	GOP/ image	10.031	29.985	83.248	168.270
dpu array efficiency	%	87.4	95.5	94.9	98.2
achieved compute	GOP/s	126.0	454.2	1132.9	2057.4

conv modules	count	57	57	77	97
dpu ops	count	244	244	310	376
dpu subgraphs	count	1	1	1	1
cpu ops		fix2float x3	fix2float x3	fix2float x3	fix2float x3
fpga area LUT	count	n/a, see §8	n/a, see §8	n/a, see §8	n/a, see §8
fpga area FF	count	n/a, see §8	n/a, see §8	n/a, see §8	n/a, see §8
fpga area BRAM	count	n/a, see §8	n/a, see §8	n/a, see §8	n/a, see §8
fpga area URAM	count	n/a, see §8	n/a, see §8	n/a, see §8	n/a, see §8
fpga area DSP	count	n/a, see §8	n/a, see §8	n/a, see §8	n/a, see §8
dpu fingerprint		433190037845252193	433190037845252193	433190037845252193	433190037845252193

Appendix B. Reproducing this

Build and check a model, from the repository root:

```
cd Yolo_v8_Versal_Implementation
./v8_run.sh python -u v8_quantize.py --weights yolov8n.pt --act hardswish \
    --quant_mode calib
./v8_run.sh python -u v8_quantize.py --weights yolov8n.pt --act hardswish \
    --quant_mode test
./v8_compile.sh yolov8n hardswish          # compiles, then reports the subgraph split
./deploy_to_board.sh                      # copies host code and xmodels across
```

Then on the board:

```
python3 board_eval_vck190.py --arch v8 \
    --model yolov8n_hardswish_vck190.xmodel \
    --images val2017 --annotations ann/instances_val2017.json \
    --out-json preds_yolov8n_hs.json --out-csv metrics_yolov8n_hs.csv

python3 measure_power.py --model yolov8n_hardswish_vck190.xmodel \
    --images val2017 --benchmark 400 --reference-fps 14.38
```

The `--arch v8` flag is required; without it the harness uses the anchor-based YOLOv3 decode and rejects the 144-channel tensors. Letterbox is the default for v8.

Plots and tables are regenerated with:

```
python tools/board_curves.py <annotations> <predictions> <outdir> <title>
python tools/extract_pr_f1.py <annotations> <predictions>
python tools/export_v8_metrics.py
python tools/report_figures.py
python tools/build_report.py
```

Appendix C. Files

Path	Contents
board_results/metrics/	per-model and combined metric CSVs, each row with its source
board_results/plots_yolov8*/	PR, P, R, F1 curves and confusion matrix per model
board_results/plots_summary/	cross-model summary charts
board_results/preds_*.json	raw COCO predictions from the board, all 5000 images
report/figures/	figures used in this document
compiled/	compiled xmodels, one directory per model and activation
quantize_result/	calibration output and INT8 xmodels

--	--